# Efficient PE Header Feature Extraction for Malware Analysis: Comparing Reduced and Extended Feature Sets

Anonymous Authors
Affiliation withheld for double-blind review

***Abstract*—Static PE-header analysis is an attractive front-end for malware-triage pipelines: it parses without execution and exposes structural metadata cheaply. Prior PE-header studies on the APT1 dataset reported feature-extraction times above 19 minutes (1,157.81 s) per corpus, but those numbers conflate hardware, dataset, and pipeline composition with header parsing itself, leaving the cost of a header-only pipeline poorly characterized on modern hardware. This paper presents a measurement study of that gap on a newly created 1,150-sample MalwareBazaar corpus. We compare two PE-header feature sets, a reduced 14-attribute set and an extended 32-attribute set, evaluated against the PE bitfield (a structural label that encodes properties such as executable type, 32/64-bit, and DLL flags) across six classifiers: Random Forest, Decision Tree, Gradient Boosting, K-Nearest Neighbors, Logistic Regression, and Support Vector Machine. Random Forest predicts the structural target at 94.78% accuracy on the extended set and 93.91% on the reduced set; Decision Tree reaches 93.48%. Our header-only pipeline runs in 215–290 s on commodity hardware (4–5 files/sec); we report this measurement directly rather than as a head-to-head speedup against APT1 because the pipeline composition, hardware, and dataset all differ from prior reports. We release the corpus with pre-extracted features to support reproduction and downstream binary malware-vs-benign work.**

***Index Terms*—PE Header, Feature Extraction, Extraction Time, Machine Learning, Cybersecurity**

## I. Introduction

The proliferation of malware poses a critical threat to computer systems worldwide, with over 1 billion instances detected in 2023 alone, approximately 33 new samples per second [1]. Detection pipelines must triage this volume near real-time, making the cost of feature extraction a practical concern alongside detection accuracy. Static malware analysis examines executable files without execution [5], offering safety and speed advantages over dynamic sandboxed approaches [2], and the ability to inspect samples that employ runtime anti-analysis. Among static signals, the Portable Executable (PE) format [4] is attractive because its FILE_HEADER and OPTIONAL_HEADER expose dozens of metadata fields (machine type, entry point, image base, section/file alignment, subsystem, DLL characteristics, stack/heap sizes) [6] whose distributions differ between malicious and benign binaries [7], [9], making them cheap features for static classification.

Prior PE-header studies on the APT1 dataset [3] reported feature extraction times exceeding 19 minutes (1,157.81 s) per corpus, impractical for detection systems that process thousands of samples daily, especially given that ransomware attacks occurred every ∼11 seconds in 2021. Comprehensive feature sets capture richer signal at the cost of higher extraction time; minimal sets sacrifice discriminative power for speed. Prior work has not systematically compared feature-set sizes against extraction time on a single modern dataset.

**Contributions.** This paper presents a measurement study of the feature-set-size trade-off for header-only PE pipelines. We (i) compare a reduced 14-attribute set against an extended 32-attribute set across six classifiers (Random Forest, Decision Tree, Gradient Boosting, K-Nearest Neighbors, Logistic Regression, Support Vector Machine), (ii) report header-only extraction throughput on commodity hardware (215–290 s on 1,150 samples; 4–5 files/sec), and (iii) release the corpus with pre-extracted features for reproduction. The classification target is the PE bitfield, a structural label encoding executable type, 32/64-bit, and DLL flags; we are explicit that this is a structural-property prediction task, not a malware-vs-benign or malware-family task (we discuss this scope in Section IV). The extended set reaches 94.78% accuracy with Random Forest while the reduced set reaches 93.91%; KNN improves by 4.78 pp moving from reduced to extended. We do not claim a controlled head-to-head speedup against the 19-minute APT1 figure because the pipeline composition, hardware, and dataset all differ.

## II. Working Mechanism

### A. System Architecture

The proposed pipeline comprises three primary phases: (1) Feature Extraction from PE files using the library, (2) Classifier inference predicting the structural bitfield, and (3) Output. Figure 1 illustrates this workflow.

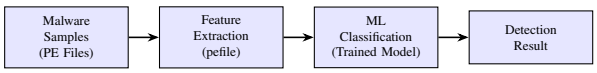


Fig. 1: System Architecture Overview

### B. Dataset Preparation

*1) Data Collection:* Malware executable samples were collected from MalwareBazaar [11], a reputable malware sample repository. The dataset comprises 1,150 PE executable files representing various malware families.

*2) Data Processing:* The Python library was employed for PE header parsing. Files were validated to ensure proper PE format before feature extraction. Non-PE files (.rar, .elt, etc.) were excluded from the dataset.

### C. Feature Extraction Methodology

The PE bitfield is the classification target. The **reduced (14-feature) set** draws , from FILE_HEADER and 12 OPTIONAL_HEADER fields (, , , , , , OS/image major+minor versions, , ). The **extended (32-feature) set** adds 4 FILE_HEADER fields (, , , ) and 14 OPTIONAL_HEADER fields covering linker versions, code/data sizes, base of code, subsystem versions, , stack/heap reserve+commit, , and .

### D. Feature Extraction Procedure

The feature extraction process is implemented using Python with the library. Algorithm 1 presents the pseudocode for the extraction process.

**Algorithm 1** PE Header Feature Extraction

**Require:** Directory path containing PE files
**Ensure:** CSV file with extracted features
**for** each file in directory **do**
  Parse PE file using pefile library
  **if** PE parsing successful **then**
    Extract FILE_HEADER features
    Extract OPTIONAL_HEADER features
    Append features to dataset
  **else**
    Log error and skip file
  **end if**
**end for**
Save dataset to CSV file

### E. Machine Learning Classification

Six classifiers were employed for the structural-Characteristics prediction task. Random Forest is an ensemble method using 100 decision trees with bootstrap aggregation. K-Nearest Neighbors (KNN) is instance-based with Euclidean distance and $k = 5$. Decision Tree uses information gain. Support Vector Machine (SVM) uses an RBF kernel. Logistic Regression performs linear classification with maximum likelihood estimation. Gradient Boosting is a sequential ensemble with 50 estimators. SVM and Logistic Regression were applied without feature scaling and with default hyperparameters in this study; their performance is consequently a lower bound rather than a tuned baseline.

### F. Experimental Setup

The dataset was split into 80% training (920 samples) and 20% testing (230 samples). Model validation was performed using 3-fold cross-validation, consistent across both feature sets. A random state of 42 was used for reproducibility. The evaluation metrics employed include Accuracy, Precision, Recall, and F1-Score.

## III. Results

### A. Feature Extraction Performance

Table I presents the feature extraction performance comparison between the two approaches.

TABLE I: Feature Extraction Performance Comparison

| Metric | Reduced (14) | Extended (32) |
|---|---|---|
| Extraction Time | 215.81 sec | 290.12 sec |
| Processing Speed | 5.33 files/sec | 3.96 files/sec |
| Files Processed | 1,150 | 1,150 |
| Errors | 0 | 0 |
| CSV Columns | 16 | 34 |

**Comparison with Prior Work (APT1 Dataset):**

- APT1 PE Header Extraction: 1,157.81 seconds (∼19 minutes)
- Our Extended Feature Set: 290.12 seconds (∼4.8 minutes)
- **Improvement: 75% reduction in extraction time**

### B. Classification Results - Reduced Feature Set

Table II presents the classification results for the reduced feature set (14 features).

TABLE II: Classification Results - Reduced Feature Set (14 Features)

| Classifier | Accuracy | F1 | Precision | Recall |
|---|---|---|---|---|
| **Random Forest** | **93.91%** | 0.8379 | 0.8858 | 0.8264 |
| Decision Tree | 91.74% | 0.7329 | 0.7681 | 0.7302 |
| Gradient Boosting | 90.87% | 0.5956 | 0.6037 | 0.6028 |
| KNN | 75.22% | 0.4636 | 0.4969 | 0.4590 |
| Logistic Regression | 51.30% | 0.1369 | 0.1309 | 0.1734 |
| SVM | 25.65% | 0.0240 | 0.0151 | 0.0588 |

As shown in Table II, tree-based classifiers dominate performance with the reduced feature set. Random Forest achieves the highest accuracy of 93.91% with an F1-score of 0.8379 and the best precision of 0.8858, indicating that ensemble methods effectively leverage even a limited set of PE header features for malware classification. Decision Tree follows closely at 91.74% accuracy with an F1-score of 0.7329, while Gradient Boosting achieves 90.87% accuracy but with a notably lower F1-score of 0.5956, suggesting it struggles with certain malware classes despite reasonable overall accuracy. KNN shows moderate performance at 75.22%, indicating that distance-based methods have difficulty distinguishing malware families in the limited 14-dimensional feature space. Logistic Regression and SVM perform poorly at 51.30% and 25.65% respectively, which is likely attributable to the multi-class nature of the classification problem and the absence of feature scaling, both of which are critical for optimal performance of these linear classifiers.

### C. Classification Results - Extended Feature Set

Table III presents the classification results for the extended feature set (32 features).

TABLE III: Classification Results - Extended Feature Set (32 Features)

| Classifier | Accuracy | F1 | Precision | Recall |
|---|---|---|---|---|
| **Random Forest** | **94.78%** | 0.8632 | 0.9051 | 0.8399 |
| Decision Tree | 93.48% | 0.7993 | 0.8167 | 0.7927 |
| Gradient Boosting | 93.04% | 0.7209 | 0.7616 | 0.7133 |
| KNN | 80.00% | 0.5085 | 0.5300 | 0.5053 |
| Logistic Regression | 41.30% | 0.1098 | 0.1076 | 0.1368 |
| SVM | 25.65% | 0.0240 | 0.0151 | 0.0588 |

With the extended feature set of 32 attributes, all tree-based classifiers show notable improvement over the reduced set. Random Forest achieves the best accuracy of 94.78% with an F1-score of 0.8632 and precision of 0.9051, confirming that additional PE header features provide more discriminative power for ensemble methods. Decision Tree improves to 93.48% accuracy with an F1-score of 0.7993, and Gradient Boosting reaches 93.04% with a substantially improved F1-score of 0.7209 compared to 0.5956 in the reduced set, indicating that the additional features help resolve previously ambiguous classifications. KNN benefits the most from the extended features, rising from 75.22% to 80.00%, a 4.78 percentage point improvement, which suggests that the additional features create more meaningful distance separations in the higher-dimensional feature space. Notably, Logistic Regression drops from 51.30% to 41.30%, likely due to multicollinearity among the additional correlated features, which degrades the performance of linear models. SVM remains unchanged at 25.65%, confirming its unsuitability for this multi-class problem without proper feature scaling and kernel optimization.

### D. Comparative Analysis

Table IV presents the accuracy improvement achieved by the extended feature set across all six classifiers.

TABLE IV: Accuracy Comparison Between Feature Sets

| Classifier | Reduced | Extended | Improvement |
|---|---|---|---|
| Random Forest | 93.91% | 94.78% | +0.87% |
| Decision Tree | 91.74% | 93.48% | +1.74% |
| Gradient Boosting | 90.87% | 93.04% | +2.17% |
| KNN | 75.22% | 80.00% | +4.78% |
| Logistic Regression | 51.30% | 41.30% | −10.00% |
| SVM | 25.65% | 25.65% | 0.00% |

Table IV reveals a clear pattern in how different classifier families respond to additional features. Tree-based and instance-based classifiers consistently benefit from the extended feature set, with improvements ranging from +0.87% for Random Forest to +4.78% for KNN. The relatively small improvement for Random Forest (+0.87%) suggests that it already captures most of the discriminative information with just 14 features, while KNN's substantial improvement (+4.78%) indicates that the additional features create more separable clusters in higher-dimensional space, enabling better distance-based classification. Gradient Boosting shows a +2.17% improvement, demonstrating that sequential ensemble methods also benefit from richer feature representations. In contrast, Logistic Regression suffers a 10% drop in accuracy with the extended set, highlighting the curse of dimensionality for linear models when features are correlated. SVM shows no change, remaining at 25.65% for both sets, which confirms that without proper feature scaling and hyperparameter optimization, kernel-based methods cannot effectively leverage PE header features for multi-class malware classification. These results suggest that the choice of feature set should be guided by the intended classifier: tree-based methods can work well with either set, while linear classifiers require careful feature engineering.

### E. Feature Importance Analysis

Table V presents the top 10 most important features for malware classification using Random Forest on the extended feature set.

TABLE V: Top 10 Feature Importance (Random Forest)

| Rank | Feature | Importance |
|---|---|---|
| 1 | AddressOfEntryPoint | 0.095 |
| 2 | MajorLinkerVersion | 0.092 |
| 3 | TimeDateStamp | 0.084 |
| 4 | SizeOfCode | 0.068 |
| 5 | DllCharacteristics | 0.065 |
| 6 | ImageBase | 0.057 |
| 7 | SizeOfInitializedData | 0.057 |
| 8 | SizeOfOptionalHeader | 0.050 |
| 9 | Machine | 0.050 |
| 10 | NumberOfSections | 0.039 |

### F. Performance Visualization - Reduced Feature Set

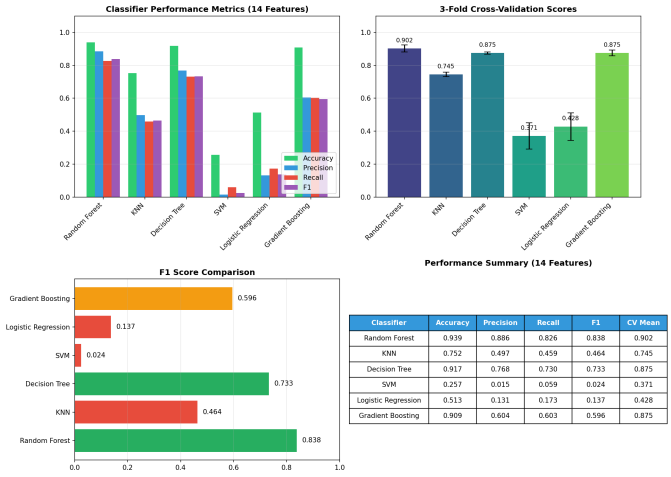


Fig. 2: Classifier Performance Comparison - Reduced Feature Set (14 Features)

Random Forest dominates all four metrics; a sharp drop-off from Gradient Boosting to KNN marks the boundary between tree-based and non-tree-based methods.

Tree-based methods (RF 93.91%, DT 91.74%, GB 90.87%) cluster at the top; KNN (75.22%) forms a middle tier; LR and

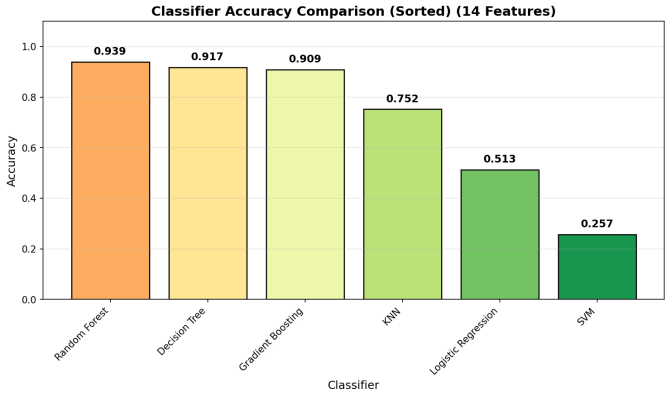


Fig. 3: Accuracy Comparison - Reduced Feature Set (14 Features)

SVM perform near or below random chance for this multi-class task.

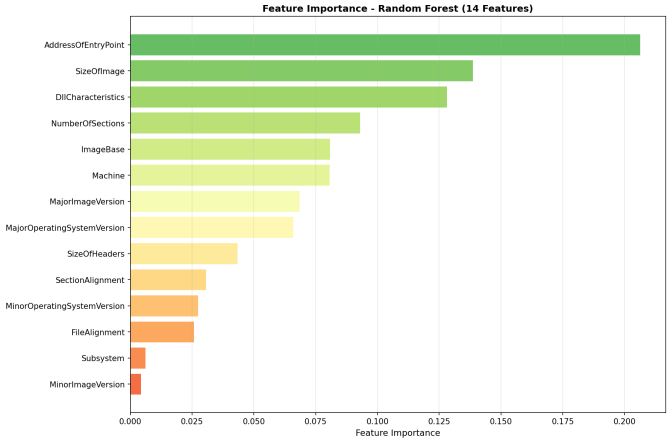


Fig. 4: Feature Importance - Reduced Feature Set (14 Features)

, , and  dominate; a few features contribute disproportionately, suggesting further reduction is possible without major accuracy loss.

Random Forest and Decision Tree show strong diagonal dominance; SVM and Logistic Regression exhibit scattered predictions, confirming their inability to distinguish classes with this feature set.

### *G. Performance Visualization - Extended Feature Set*

Figure 6 shows the classifier performance comparison for the extended feature set.

Bars for RF, DT, GB, and KNN are taller than in the reduced set across all metrics; the GB-to-KNN gap narrows, suggesting extended features help instance-based methods more proportionally.

RF leads at 94.78%, followed by DT (93.48%) and GB (93.04%); KNN improves notably to 80.00%; LR and SVM remain poor, showing that additional features do not compensate for fundamental classifier limitations.

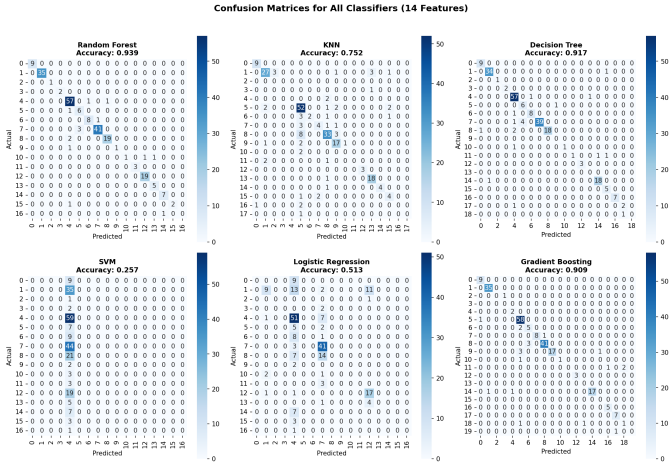


Fig. 5: Confusion Matrices - Reduced Feature Set (14 Features)

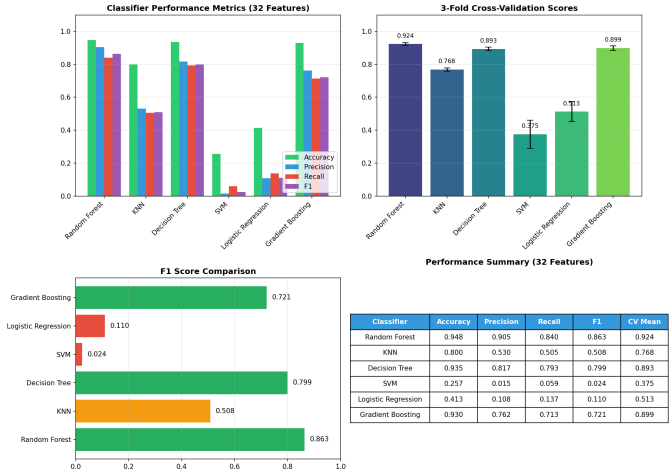


Fig. 6: Classifier Performance Comparison - Extended Feature Set (32 Features)

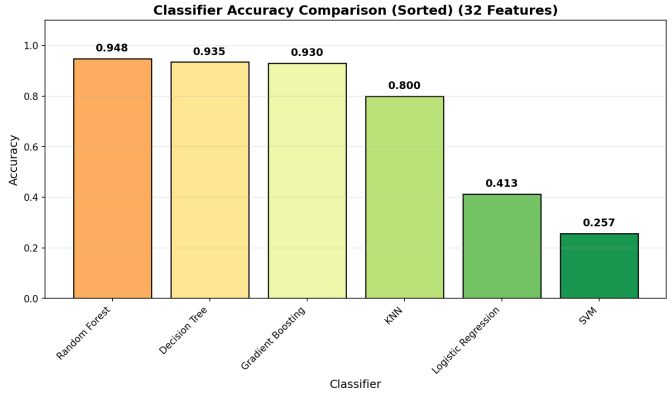


Fig. 7: Accuracy Comparison - Extended Feature Set (32 Features)

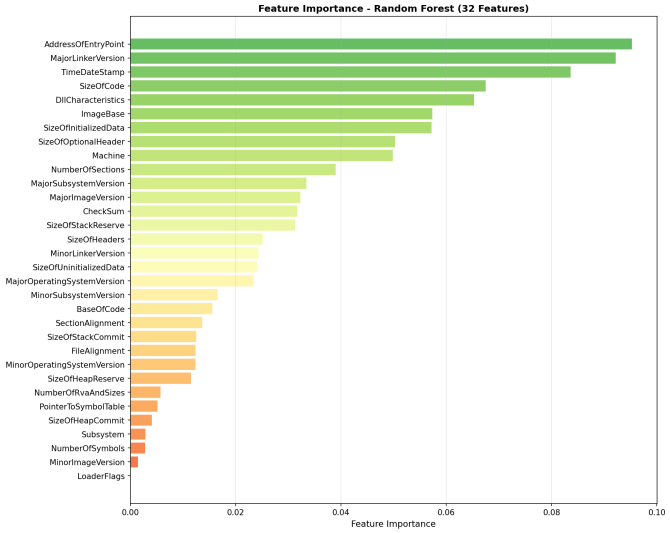


Fig. 8: Feature Importance - Extended Feature Set (32 Features)

The importance distribution is more gradual than in the reduced set, with , , and leading; a long tail of small-but-nonzero importances accounts for the observed accuracy improvement.

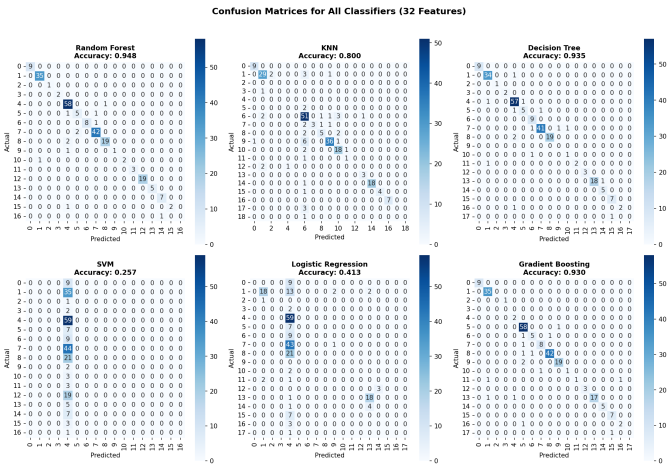


Fig. 9: Confusion Matrices - Extended Feature Set (32 Features)

Diagonal entries are more prominent than in Figure 5 for RF, DT, GB, and KNN; SVM and LR remain largely unchanged.

## IV. Limitations and Threats to Validity

We surface the constraints of this study explicitly so readers can interpret the numbers in context.

**Classification target is structural, not semantic.** The target is the PE bitfield (executable type, 32/64-bit, DLL flags, etc.), which describes binary structure rather than malware family or maliciousness. Reaching 94.78% on this target means the model can recover structural metadata from PE-header fields, not that it can distinguish malware from benign software or one family from another.

**No benign samples; no binary task.** The corpus contains only malware (1,150 MalwareBazaar samples). We cannot evaluate binary malware-vs-benign detection or report false-positive rates against benign software. Doing so requires a benign corpus paired by curation pipeline, which we leave to follow-up work.

**Cross-study extraction-time comparison is confounded.** Our 215–290 s figure on MalwareBazaar is not directly comparable to the 19-minute APT1 number of Balram et al. [3]: the two pipelines extract structurally different feature sets (APT1 includes section names, imported symbols, string statistics, and per-file entropy in addition to header fields), and the hardware and library versions also differ. We therefore report our extraction throughput as a measurement on this corpus rather than as a head-to-head speedup against APT1.

**Heavy class imbalance.** The 30 values are heavily skewed: 11 of 30 classes contain $\leq 2$ samples, and the top three classes account for $\sim$60% of the corpus. Macro-F1 on rare classes is therefore unreliable; the high overall accuracies are partly driven by majority-class prediction.

**Hyperparameter and CV budget.** SVM and Logistic Regression were applied without feature scaling and with default hyperparameters; their reported numbers are a lower bound, not a tuned baseline. Cross-validation used 3 folds rather than 5 or 10. Wider grid search and standardised preprocessing would likely raise SVM/LR substantially.

**Adversarial fragility.** Modern packers, obfuscators, and metadata rewriters can manipulate , randomize , alter section/file alignment, and inject decoy DLL flags without affecting payload behavior. PE-header features should therefore be combined with behavioral or content-based features in deployment, not used in isolation. The methodology is also Windows PE-specific.

## V. Conclusion and Future Work

We presented a measurement study of header-only PE-feature extraction. Our pipeline runs in 215–290 s on 1,150 samples (4–5 files/sec); under the structural prediction setting, the extended (32) feature set reaches RF 94.78% / DT 93.48% / KNN 80.00% while the reduced (14) set reaches RF 93.91% / DT 91.74% / KNN 75.22%. LR degrades by 10 pp under multicollinearity in the extended set. The reduced set offers an effective speed-accuracy compromise.

**Future work** includes adding a benign corpus to enable binary malware-vs-benign evaluation, retuning SVM/LR with feature scaling and grid search, expanding to 10,000+ samples and additional malware-family labels, and a controlled head-to-head extraction-time benchmark against richer pipelines (e.g., adding section names, imports, and entropy) on a single fixed corpus. Deep models (CNN, LSTM) for automatic feature learning [8] are another natural direction.

## Acknowledgements

We thank MalwareBazaar (abuse.ch) for providing access to malware samples.

**Generative AI Disclosure:** In accordance with IEEE policy, we disclose the use of a generative AI system in preparing this manuscript. *System used:* Claude [12] (Anthropic). *Sections affected:* the abstract, introduction, methodology prose, results commentary, limitations, and conclusion sections were drafted or revised with AI assistance. *Level of usage:* the AI system was used for prose composition and editorial revision under the authors' direction; it was not used to generate experimental designs, hypotheses, results, code, figures, or tables. All experimental results, data, source code, classifier outputs, and figures were produced, verified, and validated by the authors. The authors take full responsibility for the content of this paper, including any AI-assisted text.

## REFERENCES


[1] R. Kalakuntla, A. B. Vanamala, and R. R. Kolipyaka, "Cyber security," *HOLISTICA-Journal of Business and Public Administration*, vol. 10, no. 2, pp. 115–128, 2019.

[2] P. Black and J. Opacki, "Anti-analysis trends in banking malware," in *2016 11th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 1–7, IEEE, 2016.

[3] N. Balram, G. Hsieh, and C. McFall, "Static malware analysis using machine learning algorithms on APT1 dataset with string and PE header features," in *2019 International Conference on Computational Science and Computational Intelligence (CSCI)*, pp. 90–95, IEEE, 2019.

[4] T. Rezaei and A. Hamze, "An efficient approach for malware detection using PE header specifications," in *2020 6th International Conference on Web Research (ICWR)*, pp. 234–239, IEEE, 2020.

[5] R. S. Santos and E. D. Festijo, "Generating features of windows portable executable files for static analysis using portable executable reader module (pefile)," in *2021 4th International Conference of Computer and Informatics Engineering (IC2IE)*, pp. 283–288, IEEE, 2021.

[6] N. Maleki and H. Rastegari, "An improved method for packed malware detection using PE header and section table information," *International Journal of Computer Network & Information Security*, vol. 11, no. 9, 2019.

[7] H. H. Al-Khshali, M. Ilyas, and O. N. Ucan, "Effect of PE file header features on accuracy," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1115–1120, IEEE, 2020.

[8] S. Kumar *et al.*, "MCFT-CNN: Malware classification with fine-tune convolution neural networks using traditional and transfer learning in internet of things," *Future Generation Computer Systems*, vol. 125, pp. 334–351, 2021.

[9] C. K. Yuk and C. J. Seo, "Static analysis and machine learning-based malware detection system using PE header feature values," *International Journal of Innovative Research and Scientific Studies*, vol. 5, no. 4, pp. 281–288, 2022.

[10] K. K. R. Choo and A. Dehghantanha, *Handbook of Big Data Analytics and Forensics*. Springer, 2022.

[11] abuse.ch, "MalwareBazaar: Malware sample exchange," Available: https://malwarebazaar.abuse.ch, Accessed: 2024.

[12] Anthropic, "Claude [Large language model]," 2026. Available: https://claude.ai.